

Next, we load the length of `v` into a register and check that index `k` is OK.

```
LDUR X11,[X0,#8]      // Temp reg X11 = length of array v
CMP X1,XZR             // Compare k to 0
B.LT IndexOutOfBounds // if k < 0, goto Error
CMP X1,X11             // Compare k to length
B.GE IndexOutOfBounds // if k >= length, goto Error
```

This check is followed by a check that `k+1` is within bounds.

```
ADDI X10,X1,#1        // Temp reg X10 = k+1
CMP X10,XZR           // Compare k+1 to 0
B.LT IndexOutOfBounds // if k+1 < 0, goto Error
CMP X10,X11           // Compare k+1 to length
B.GE IndexOutOfBounds // if k+1 >= length, goto Error
```

Figure e2.15.11 highlights the extra LEGv8 instructions in `swap` that a Java compiler might produce. We again must adjust the offset in the load and store to account for two doublewords reserved for the method table and length.

Figure e2.15.12 shows the method body for those new instructions for `sort`. (We can take the saving, restoring, and return from Figure e2.28.)

The first test is again to make sure the pointer to `v` is not null:

```
CBZ X0,NullPointer    // if X0==0,goto Error
```

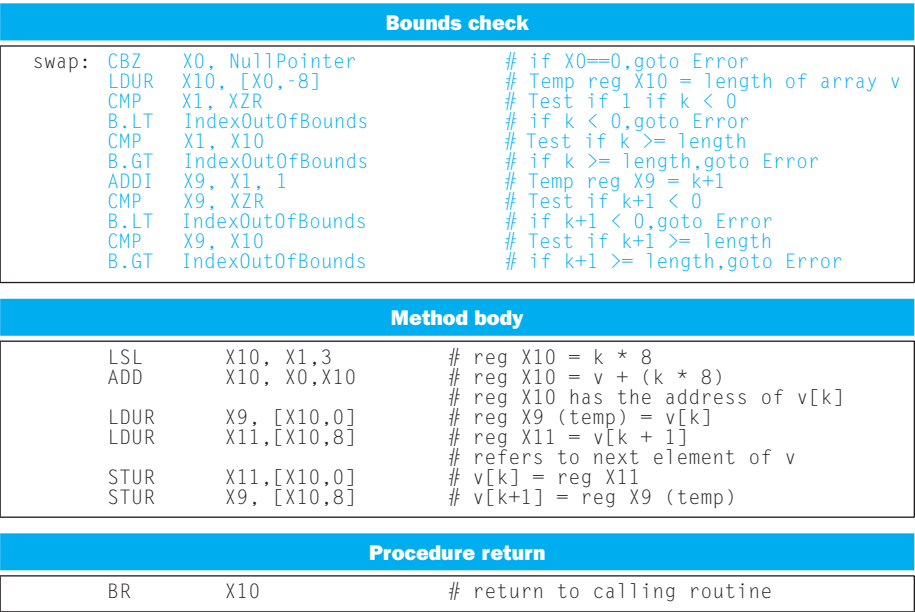


FIGURE e2.15.11 LEGv8 assembly code of the procedure `SWaP` in Figure e2.24.

Method body			
Move parameters	MOV	X21, X0	# copy parameter X0 into X21
Test ptr null	CBZ	X0, NullPointer	# if X0==0, goto Error
Get array length	LDUR	X22, [X0,8]	# X22 = length of array v
Outer loop	for1tst: MOV CMP B.GT	X19, XZR X19, X1t0, exit1	# i = 0 # test if X19 ≥ X1 (i ≥ n) # go to exit1 if X19 ≥ X1 (i ≥ n)
Inner loop start	for2tst: SUBI CMP B.LT	X20, X19, 1 X20, XZR exit2	# j = i - 1 # Test if X20 < 0. (j < 0) # go to exit2 if X20 < 0 (j < 0)
Test if j too big	CMP B.GT	X20, X22 IndexOutOfBounds	# Test if j >= length # if j >= length, goto Error
Get v[j]	LSL ADD LDUR	X10, X20, 3 X11, X0, X10 X12, [X11,0]	# reg X10 = j * 8 # reg X11 = v + (j * 8) # reg X12 = v[j]
Test if j+1 < 0 or if j+1 too big	ADDI CMP B.LT CMP B.GT	X10, X20, 1 X10, XZR IndexOutOfBounds X10, X22 IndexOutOfBounds	# Temp reg X10 = j+1 # Test if j+1 < 0 # if j+1 < 0, goto Error # Test if j+1 >= length # if j+1 >= length, goto Error
Get v[j+1]	LDUR	X13, [X11,8]	# reg X13 = v[j + 1]
Load method table	LDUR	X14, [X0,0]	# X14 = address of method table
Get method addr	LDUR	X14, [X14,16]	# X14 = address of first method
Pass parameters	MOV MOV	X0, X21 X1, X20	# 1st parameter of compareTo is v[j] # 2nd param. of compareTo is v[j+1]
Set return addr	LDA	X30, L1	# load return address
Call indirectly	BR	X14	# call code for compareTo
Test if should skip swap	L1: CMP B.LE	X12, X13 exit2	# compare X12 to X13 # go to exit2 if X12 ≤ X13
Pass parameters and call swap	MOV MOV BL	X0, X21 X1, X20 swap	# 1st parameter of swap is v # 2nd parameter of swap is j # swap code shown in Figure 2.34
Inner loop end	SUBI B	X20, X20, 1 for2tst	# j -= 1 # jump to test of inner loop
Outer loop	exit2: ADDI B	X19, X19, 1 for1tst	# i += 1 # jump to test of outer loop

FIGURE e2.15.12 LEGv8 assembly version of the method body of the Java version of `Sort`. The new code is highlighted in this figure. We must still add the code to save and restore registers and the return from the LEGv8 code found in [Figure e2.27](#). To keep the code similar to that figure, we load `v.length` into X22 instead of into a temporary register. To reduce the number of lines of code, we make the simplifying assumption that `compareTo` is a leaf procedure and we do not need to push registers to be saved on the stack.

Next, we load the length of the array (we use register X22 to keep it similar to the code for the C version of `swap`):

```
LDUR    X22, [X0, #8]    //X22 = length of array v
```

Now we must ensure that the index is within bounds. Since the first test of the inner loop is to test if j is negative, we can skip that initial bound test. That leaves the test for too big:

```
CMP      X20,X22          // compare j to length
B.GE,IndexOutOfBounds    //if j >= length, goto Error
```

The code for testing $j + 1$ is quite similar to the code for checking $k + 1$ in `swap`, so we skip it here.

The key difference is the invocation of `compareTo`. We first load the address of the table of legal methods, which we assume is two doublewords before the beginning of the array:

```
LDUR    X14, [X0, #0]    // X14 = address of method table
```

Given the address of the method table for this object, we then get the desired method. Let's assume `compareTo` is the third method in the `Comparable` class. To pick the address of the third method, we load that address into a temporary register:

```
LDUR    X14, [X14, #16]   // X14 = address of third method
```

We are now ready to call `compareTo`. The next step is to save the necessary registers on the stack. Fortunately, we don't need the temporary registers or argument registers after the method invocation, so there is nothing to save. Thus, we simply pass the parameters for `compareTo`:

```
MOV    X0, X12    // 1st parameter of compareTo is v[j]
MOV    X1, X13    // 2nd parameter of compareTo is v[j+1]
```

Since we are using a branch register to invoke `compareTo`, we need to pass the return address explicitly. We use the pseudoinstruction load address (LDA) and label where we want to return, and then do the indirect branch:

```
LDA    X30, L1    // load return address
BR     X14        // to code for compareTo
```

The method returns, with `X6` determining which of the two elements is larger. If $X6 > 0$, then $v[j] > v[j+1]$, and we need to swap. Thus, to skip the swap, we need to test if $X6 \leq 0$. We also need to include the label for the return address:

```
L1:    CMP    X6, XZR    // test if X6 ≤ 0
        B.LE  exit2     // go to exit2 if v[j] ≤ v[j+1]
```

The LEGv8 code for `compareTo` is left as an exercise.

Hardware/ Software Interface

The main changes for the Java versions of `sort` and `swap` are testing for null object references and index out-of-bounds errors, and the extra method invocation to give a more general compare. This method invocation is more expensive than a C procedure call, since it requires a load, a conditional branch, a pair of chained loads, and an indirect branch. As we see in [Chapter 4](#), dependent loads and indirect branches can be relatively slow on modern processors. The increasing popularity of Java suggests that many programmers today are willing to leverage the high performance of modern processors to pay for error checking and code reuse.

Elaboration Although we test each reference to j and $j + 1$ to be sure that these indices are within bounds, an assembly language programmer might look at the code and reason as follows:

1. The inner *for* loop is only executed if $j \leq 0$ and since $j + 1 > j$, there is no need to test $j + 1$ to see if it is less than 0.
2. Since i takes on the values, 0, 1, 2, ..., ($\text{data.length} - 1$) and since j takes on the values $i - 1$, $i - 2$, ..., 2, 1, 0, there is no need to test if $j \leq \text{data.length}$ since the largest value j can be is $\text{data.length} - 2$.
3. Following the same reasoning, there is no need to test whether $j + 1 \leq \text{data.length}$ since the largest value of $j + 1$ is $\text{data.length} - 1$.

There are coding tricks in [Chapter 2](#) and superscalar execution in [Chapter 4](#) that lower the effective cost of such bounds checking, but only high optimizing compilers can reason this way. Note that if the compiler inlined the swap method into sort, many checks would be unnecessary.

Elaboration Look carefully at the code for swap in [Figure e2.15.11](#). See anything wrong in the code, or at least in the explanation of how the code works? It implicitly assumes that each `Comparable` element in v is 8 bytes long. Surely, you need much more than 8 bytes for a complex subclass of `Comparable`, which could contain any number of fields. Surprisingly, this code does work, because an important property of Java's semantics forces the use of the same, small representation for all variables, fields, and array elements that belong to `Comparable` or its subclasses.

Java types are divided into *primitive types*—the predefined types for numbers, characters, and Booleans—and *reference types*—the built-in classes like `String`, user-defined classes, and arrays. Values of reference types are pointers (also called *references*) to anonymous objects that are themselves allocated in the heap. For the programmer, this means that assigning one variable to another does not create a new object, but instead makes both variables refer to the same object. Because these objects are anonymous, and programs therefore have no way to refer to them directly, a program must use indirection through a variable to read or write any objects' fields (variables). Thus, because the data structure allocated for the array v consists entirely of pointers, it is safe to assume they are all the same size, and the same swapping code works for all of `Comparable`'s subtypes.

To write sorting and swapping functions for arrays of primitive types requires that we write new versions of the functions, one for each type. This replication is for two reasons. First, primitive type values do not include the references to dispatching tables that we used on `Comparables` to determine at runtime how to compare values. Second, primitive values come in different sizes: 1, 2, 4, or 8 bytes.

The pervasive use of pointers in Java is elegant in its consistency, with the penalty being a level of indirection and a requirement that objects be allocated on the heap. Furthermore, in any language where the lifetimes of the heap-allocated anonymous objects are independent of the lifetimes of the named variables, fields, and array elements that reference them, programmers must deal with the problem of deciding when it is safe to deallocate heap-allocated storage. Java's designers chose to use

garbage collection. Of course, use of garbage collection rather than explicit user memory management also improves program safety.

C++ provides an interesting contrast. Although programmers can write essentially the same pointer-manipulating solution in C++, there is another option. In C++, programmers can elect to forgo the level of indirection and directly manipulate an array of objects, rather than an array of pointers to those objects. To do so, C++ programmers would typically use the template capability, which allows a class or function to be parameterized by the type of data on which it acts. Templates, however, are compiled using the equivalent of macro expansion. That is, if we declared an instance of sort capable of sorting types X and Y, C++ would create two copies of the code for the class: one for sort<X> and one for sort<Y>, each specialized accordingly. This solution increases code size in exchange for making comparison faster (since the function calls would not be indirect, and might even be subject to inline expansion). Of course, the speed advantage would be canceled if swapping the objects required moving large amounts of data instead of just single pointers. As always, the best design depends on the details of the problem.

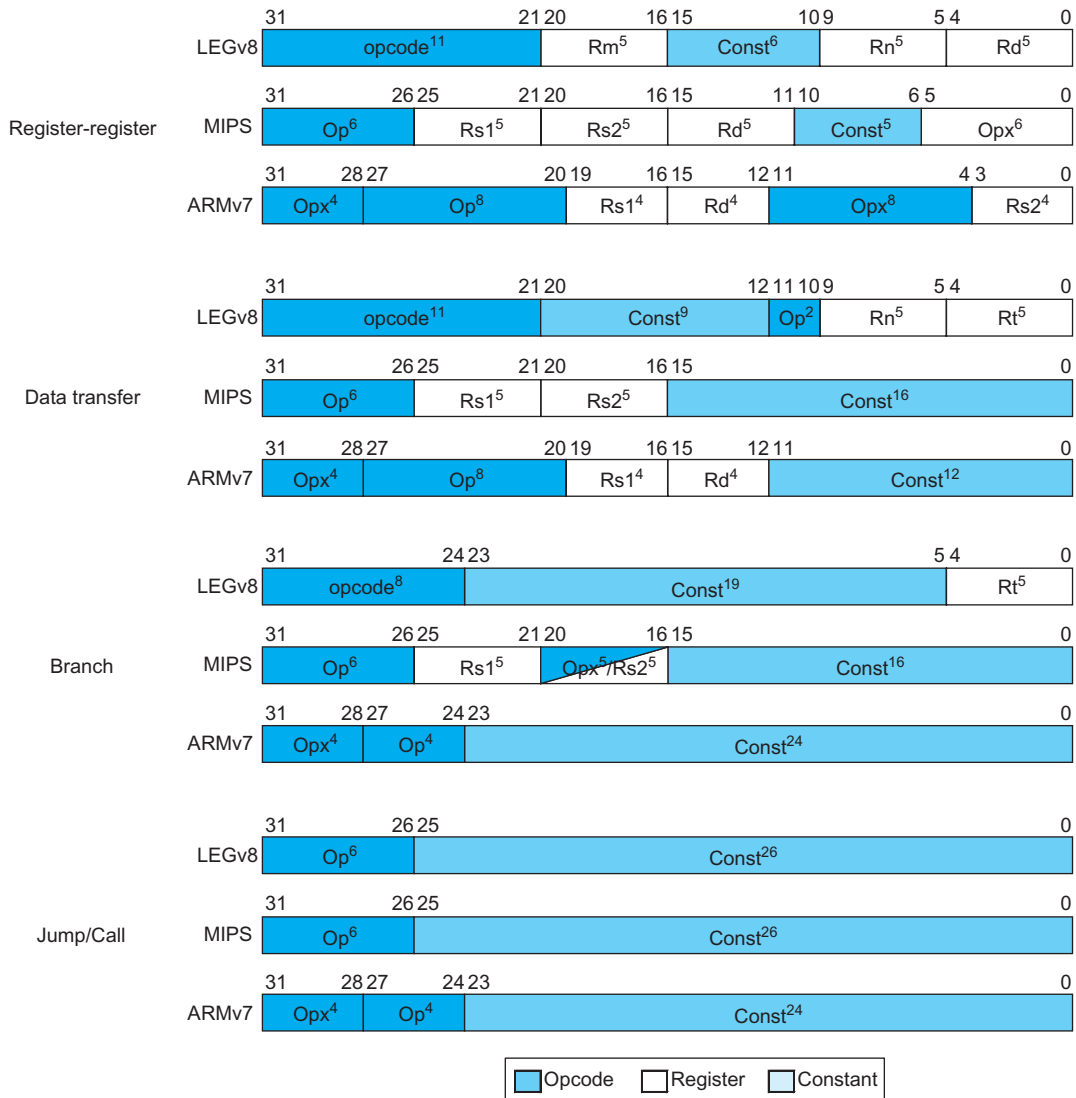


FIGURE 2.32 Instruction formats of LEGv8, MIPS, and ARMv7. The differences result in part from whether the architecture has 16 registers (ARMv7) or 32 registers (ARMv8 and MIPS).

senior than ARMv8. The good news is that if you know ARMv8, it will be very easy to pick up MIPS. To show their similarity, [Figure 2.32](#) compares instruction formats for ARMv8 and MIPS, as well as ARMv7.

The MIPS ISA has both 32-bit address and 64-bit address versions, sensibly called MIPS-32 and MIPS-64. These instruction sets are virtually identical except

for the larger address size needing 64-bit registers instead of 32-bit registers. Here are the common features between ARMv8 and MIPS:

- All instructions are 32 bits wide for both architectures.
- Both have 32 general-purpose registers, with one register being hardwired to 0.
- The only way to access memory is via load and store instructions on both architectures.
- Unlike some architectures, there are no instructions that can load or store many registers in MIPS or ARMv8.
- Both have instructions that branch if a register is equal to zero and branch if a register is not equal to zero.
- Both have 32 floating-point registers, as we shall see in [Chapter 3](#).
- Both sets of addressing modes work for all word sizes.

One of the main differences between ARMv8 and MIPS is for conditional branches other than equal or not equal. ARMv8 and many other architectures use condition codes. MIPS instead relies on a comparison instruction that sets a register to 0 or 1 depending on whether the comparison is true. Programmers then follow that comparison instruction with a branch on equal to or not equal to zero depending on the desired outcome of the comparison. Keeping with its minimalist philosophy, MIPS only performs less than comparisons, leaving it up to the programmer to switch order of operands or to switch the condition being tested by the branch to get all the desired outcomes. MIPS has both signed and unsigned versions of the set on less than instructions: SLT and SLTU.

When we look beyond the core instructions that are most commonly used, the other main difference is that the full ARMv8 is a much larger instruction set than the complete MIPS instruction set, as we shall see in Section 2.19. That size difference means many more instruction formats, many more addressing modes, and many more operations.

2.17

Real Stuff: ARMv7 (32-bit) Instructions

Standing originally for the Acorn RISC Machine, later changed to Advanced RISC Machine, ARMv1 came out the same year as MIPS. Both architectures used 32-bit addresses, which was just fine in 1985. Many versions of the 32-bit address ARM instruction set came out over the years, culminating with ARMv7 in 2005.

ARM architects could see the writing on the wall of its 32-bit address computer and began design of the 64-bit address version of ARM in 2007. Of the many potential problems in an instruction set, the one that is almost impossible to

overcome is having too small a memory address. For example, the 16-bit address MOSTek 6502 powered the Apple II, but even given this head start with the first commercially successful personal computer, its lack of address bits condemned it to the dustbin of history.

ARMv8 was announced in 2011 with 64-bit addresses, and was finally revealed in 2013. Rather than some minor cosmetic changes to make all the registers 64 bits wide, which is basically what MIPS did earlier to create MIPS-64, ARM did a complete overhaul. Perhaps surprisingly, in our opinion the philosophy of the ARMv8 instruction ended up being much closer to MIPS than it is to ARMv7.

Here are the similarities between ARMv7 and ARMv8:

- All instructions are 32 bits wide for both architectures.
- The only way to access memory is via load and store instructions on both architectures.

Here are some of the differences:

- ARMv7 and the earlier ARM instruction sets have just 15 general-purpose registers, which can't make compiler writers happy.
- No register is hardwired to 0, so ARMv7 and its predecessors need extra instructions to perform some operations that ARMv8 can do with XZR.
- The missing 16th register in ARMv7 and its predecessors is the *program counter* (PC). Thus, programmers get unexpected branches if that register is altered as part of an arithmetic logic instruction. PC is not one of the 32 ARMv8 registers.
- ARMv7 addressing modes do not work for all data sizes, which is not the case in ARMv8.
- ARMv7 has Load Multiple and Store Multiple instructions, which allow several registers to be moved to and from memory. ARMv8 does not.
- Virtually every one of the ARMv5 (and earlier) instructions had a 4-bit conditional execution field, which turns it into a non-operating instruction (NOP) if the condition is false. Indeed, the conditional branch instruction is really just a special case of the regular branch instruction. This field is missing in many ARMv7 instructions, in part to make room for new instructions. ARMv8 has no such field.
- Rather than the immediate field simply being a constant, it is essentially an input to a function that produces a constant. The eight least-significant bits of ARMv7's 12-bit immediate field are zero-extended to a 32-bit value and then rotated right the number of bits specified in the first four bits of the field multiplied by two. The goal was to encode more useful constants in fewer bits. The full ARMv8 has its own complicated encoding for immediate fields of logical instructions. Whether this technique actually catches many more

immediates than a simple constant field—which is what MIPS uses—would be an interesting study.

- Unlike ARMv8, the early ARM instruction sets omitted a divide instruction (see [Chapter 3](#)). It is included in ARMv7, but it is optional in a compliant ARMv7 core.

2.18 Real Stuff: x86 Instructions

Beauty is altogether in the eye of the beholder.

Margaret Wolfe
Hungerford, Molly
Bawn, 1877

Designers of instruction sets sometimes provide more powerful operations than those found in ARMv8 and MIPS. The goal is generally to reduce the number of instructions executed by a program. The danger is that this reduction can occur at the cost of simplicity, increasing the time a program takes to execute because the instructions are slower. This slowness may be the result of a slower clock cycle time or of requiring more clock cycles than a simpler sequence.

The path toward operation complexity is thus fraught with peril. Section 2.20 demonstrates the pitfalls of complexity.

Evolution of the Intel x86

ARMv8 and MIPS were the vision of single groups working at the same time; the pieces of these architectures fit nicely together. Such is not the case for the x86; it is the product of several independent groups who evolved the architecture over 35 years, adding new features to the original instruction set as someone might add clothing to a packed bag. Here are important x86 milestones.

- **1978:** The Intel 8086 architecture was announced as an assembly language-compatible extension of the then-successful Intel 8080, an 8-bit microprocessor. The 8086 is a 16-bit architecture, with all internal registers 16 bits wide. Unlike ARMv8, the registers have dedicated uses, and hence the 8086 is not considered a **general-purpose register (GPR)** architecture.
- **1980:** The Intel 8087 floating-point coprocessor is announced. This architecture extends the 8086 with about 60 floating-point instructions. Instead of using registers, it relies on a stack (see [Section 2.22](#) and [Section 3.7](#)).
- **1982:** The 80286 extended the 8086 architecture by increasing the address space to 24 bits, by creating an elaborate memory-mapping and protection model (see [Chapter 5](#)), and by adding a few instructions to round out the instruction set and to manipulate the protection model.
- **1985:** The 80386 extended the 80286 architecture to 32 bits. In addition to a 32-bit architecture with 32-bit registers and a 32-bit address space, the 80386 added new addressing modes and additional operations. The expanded instructions make the 80386 nearly a general-purpose register machine. The

general-purpose register (GPR)

A register that can be used for addresses or for data with virtually any instruction.

80386 also added paging support in addition to segmented addressing (see [Chapter 5](#)). Like the 80286, the 80386 has a mode to execute 8086 programs without change.

- **1989–95:** The subsequent 80486 in 1989, Pentium in 1992, and Pentium Pro in 1995 were aimed at higher performance, with only four instructions added to the user-visible instruction set: three to help with multiprocessing (see [Chapter 6](#)) and a conditional move instruction.
- **1997:** After the Pentium and Pentium Pro were shipping, Intel announced that it would expand the Pentium and the Pentium Pro architectures with MMX (*Multi Media Extensions*). This new set of 57 instructions uses the floating-point stack to accelerate multimedia and communication applications. MMX instructions typically operate on multiple short data elements at a time, in the tradition of *single instruction, multiple data* (SIMD) architectures (see [Chapter 6](#)). Pentium II did not introduce any new instructions.
- **1999:** Intel added another 70 instructions, labeled SSE (*Streaming SIMD Extensions*) as part of Pentium III. The primary changes were to add eight separate registers, double their width to 128 bits, and add a single precision floating-point data type. Hence, four 32-bit floating-point operations can be performed in parallel. To improve memory performance, SSE includes cache prefetch instructions plus streaming store instructions that bypass the caches and write directly to memory.
- **2001:** Intel added yet another 144 instructions, this time labeled SSE2. The new data type is double precision arithmetic, which allows pairs of 64-bit floating-point operations in parallel. Almost all of these 144 instructions are versions of existing MMX and SSE instructions that operate on 64 bits of data in parallel. Not only does this change enable more multimedia operations; it gives the compiler a different target for floating-point operations than the unique stack architecture. Compilers can choose to use the eight SSE registers as floating-point registers like those found in other computers. This change boosted the floating-point performance of the Pentium 4, the first microprocessor to include SSE2 instructions.
- **2003:** A company other than Intel enhanced the x86 architecture this time. AMD announced a set of architectural extensions to increase the address space from 32 to 64 bits. Similar to the transition from a 16- to 32-bit address space in 1985 with the 80386, AMD64 widens all registers to 64 bits. It also increases the number of registers to 16 and increases the number of 128-bit SSE registers to 16. The primary ISA change comes from adding a new mode called *long mode* that redefines the execution of all x86 instructions with 64-bit addresses and data. To address the larger number of registers, it adds a new prefix to instructions. Depending how you count, long mode also adds four to 10 new instructions and drops 27 old ones. PC-relative data addressing is another extension. AMD64 still has a mode that is identical

to x86 (*legacy mode*) plus a mode that restricts user programs to x86 but allows operating systems to use AMD64 (*compatibility mode*). These modes allow a more graceful transition to 64-bit addressing than the HP/Intel IA-64 architecture.

- **2004:** Intel capitulates and embraces AMD64, relabeling it *Extended Memory 64 Technology* (EM64T). The major difference is that Intel added a 128-bit atomic compare and swap instruction, which probably should have been included in AMD64. At the same time, Intel announced another generation of media extensions. SSE3 adds 13 instructions to support complex arithmetic, graphics operations on arrays of structures, video encoding, floating-point conversion, and thread synchronization (see Section 2.11). AMD added SSE3 in subsequent chips and the missing atomic swap instruction to AMD64 to maintain binary compatibility with Intel.
- **2006:** Intel announces 54 new instructions as part of the SSE4 instruction set extensions. These extensions perform tweaks like sum of absolute differences, dot products for arrays of structures, sign or zero extension of narrow data to wider sizes, population count, and so on. They also added support for virtual machines (see [Chapter 5](#)).
- **2007:** AMD announces 170 instructions as part of SSE5, including 46 instructions of the base instruction set that adds three operand instructions like ARMv8.
- **2011:** Intel ships the Advanced Vector Extension that expands the SSE register width from 128 to 256 bits, thereby redefining about 250 instructions and adding 128 new instructions.

This history illustrates the impact of the “golden handcuffs” of compatibility on the x86, as the existing software base at each step was too important to jeopardize with significant architectural changes.

Whatever the artistic failures of the x86, keep in mind that this instruction set largely drove the PC generation of computers and still dominates the Cloud portion of the post-PC era. Manufacturing 350M x86 chips per year may seem small compared to 14 billion ARM chips, but many companies would love to control such a market. Nevertheless, this checkered ancestry has led to an architecture that is difficult to explain and impossible to love.

Brace yourself for what you are about to see! Do *not* try to read this section with the care you would need to write x86 programs; the goal instead is to give you familiarity with the strengths and weaknesses of the world’s most popular desktop architecture.

Rather than show the entire 16-bit, 32-bit, and 64-bit instruction set, in this section we concentrate on the 32-bit subset that originated with the 80386. We start our explanation with the registers and addressing modes, move on to the integer operations, and conclude with an examination of instruction encoding.

Name	31	0	Use
EAX			GPR 0
ECX			GPR 1
EDX			GPR 2
EBX			GPR 3
ESP			GPR 4
EBP			GPR 5
ESI			GPR 6
EDI			GPR 7
	CS		Code segment pointer
	SS		Stack segment pointer (top of stack)
	DS		Data segment pointer 0
	ES		Data segment pointer 1
	FS		Data segment pointer 2
	GS		Data segment pointer 3
EIP			Instruction pointer (PC)
EFLAGS			Condition codes

FIGURE 2.33 The 80386 register set. Starting with the 80386, the top eight registers were extended to 32 bits and could also be used as general-purpose registers.

x86 Registers and Data Addressing Modes

The registers of the 80386 show the evolution of the instruction set ([Figure 2.33](#)). The 80386 extended all 16-bit registers (except the segment registers) to 32 bits, prefixing an *E* to their name to indicate the 32-bit version. We'll refer to them generically as GPRs (*general-purpose registers*). The 80386 contains only eight GPRs. This means ARMv8 and MIPS programs can use four times as many and ARMv7 twice as many.

[Figure 2.34](#) shows the arithmetic, logical, and data transfer instructions are two-operand instructions. There are two important differences here. The x86 arithmetic and logical instructions must have one operand act as both a source

Source/destination operand type	Second source operand
Register	Register
Register	Immediate
Register	Memory
Memory	Register
Memory	Immediate

FIGURE 2.34 Instruction types for the arithmetic, logical, and data transfer instructions. The x86 allows the combinations shown. The only restriction is the absence of a memory-memory mode. Immediates may be 8, 16, or 32 bits in length; a register is any one of the 14 major registers in [Figure 2.33](#) (not EIP or EFLAGS).

and a destination; ARMv7, ARMv8 and MIPS allow separate registers for source and destination. This restriction puts more pressure on the limited registers, since one source register must be modified. The second important difference is that one of the operands can be in memory. Thus, virtually any instruction may have one operand in memory, unlike ARMv7, ARMv8 and MIPS.

Data memory-addressing modes, described in detail below, offer two sizes of addresses within the instruction. These so-called *displacements* can be 8 bits or 32 bits.

Although a memory operand can use any addressing mode, there are restrictions on which *registers* can be used in a mode. [Figure 2.35](#) shows the x86 addressing modes and which GPRs cannot be used with each mode, as well as how to get the same effect using ARMv8 instructions.

x86 Integer Operations

The 8086 provides support for both 8-bit (*byte*) and 16-bit (*word*) data types. The 80386 adds 32-bit addresses and data (*doublewords*) in the x86. (AMD64 adds 64-bit addresses and data, called *quad words*; we'll stick to the 80386 in this section.) The data type distinctions apply to register operations as well as memory accesses.

Almost every operation works on both 8-bit data and on one longer data size. That size is determined by the mode and is either 16 bits or 32 bits.

Clearly, some programs want to operate on data of all three sizes, so the 80386 architects provided a convenient way to specify each version without expanding code size significantly. They decided that either 16-bit or 32-bit data dominate most programs, and so it made sense to be able to set a default large size. This default data size is set by a bit in the code segment register. To override the default data size, an 8-bit *prefix* is attached to the instruction to tell the machine to use the other large size for this instruction.

The prefix solution was borrowed from the 8086, which allows multiple prefixes to modify instruction behavior. The three original prefixes override the default segment register, lock the bus to support synchronization (see [Section 2.11](#)), or repeat the following instruction until the register ECX counts down to 0. This last

Mode	Description	Register restrictions	ARMv8 equivalent
Register indirect	Address is in a register.	Not ESP or EBP	LDUR X1, [X2,#40]
Based mode with 8- or 32-bit displacement	Address is contents of base register plus displacement.	Not ESP	LDUR X1, [X2,#40] # <= 9-bit displacement
Base plus scaled index	The address is Base + (2 ^{Scale} x Index) where Scale has the value 0, 1, 2, or 3.	Base: any GPR Index: not ESP	MUL X0, X2, #8 ADD X0, X0, X1 LDUR X1, [X0,0]
Base plus scaled index with 8- or 32-bit displacement	The address is Base + (2 ^{Scale} x Index) + displacement where Scale has the value 0, 1, 2, or 3.	Base: any GPR Index: not ESP	MUL X0, X2, #8 ADD X0, X0, X1 LDUR X1, [X0,#40] // <= 9-bit displacement

FIGURE 2.35 x86 32-bit addressing modes with register restrictions and the equivalent ARMv8 code. The Base plus Scaled Index addressing mode, not found in LEGv8 or MIPS, is included to avoid the multiplies by 8 (scale factor of 3) to turn an index in a register into a byte address (see [Figures 2.26 and 2.28](#)). A scale factor of 1 is used for 16-bit data, and a scale factor of 2 for 32-bit data. A scale factor of 0 means the address is not scaled. If the displacement is longer than 9 bits in the second or fourth modes, then the ARMv8 equivalent mode would need more instructions. Either a MOVZ to load 16 bits of the displacement and an ADDI to sum the upper address with the base register X2, or a MOVZ followed by a MOVK to get 32-bits of address plus an ADDI to add this address to the base register X2. (Intel gives two different names to what is called Based addressing mode—Based and Indexed—but they are essentially identical and we combine them here.)

prefix was intended to be paired with a byte move instruction to move a variable number of bytes. The 80386 also added a prefix to override the default address size.

The x86 integer operations can be divided into four major classes:

1. Data movement instructions, including move, push, and pop.
2. Arithmetic and logic instructions, including test, integer, and decimal arithmetic operations.
3. Control flow, including conditional branches, unconditional branches, calls, and returns.
4. String instructions, including string move and string compare.

The first two categories are unremarkable, except that the arithmetic and logic instruction operations allow the destination to be either a register or a memory location. [Figure 2.36](#) shows some typical x86 instructions and their functions.

Conditional branches on the x86 are based on *condition codes* or *flags*, like ARMv7. Condition codes are set as a side effect of an operation; most are used to compare the value of a result to 0. Branches then test the condition codes. PC-relative branch addresses must be specified in the number of bytes, since unlike ARMv7, ARMv8 and MIPS, 80386 instructions are not all 4 bytes in length.

String instructions are part of the 8080 ancestry of the x86 and are not commonly executed in most programs. They are often slower than equivalent software routines (see the *Fallacy* on page 169).

[Figure 2.37](#) lists some of the integer x86 instructions. Many of the instructions are available in both byte and word formats.

Instruction	Function
je name	if equal(condition code) {EIP=name}; EIP-128 <= name < EIP+128
jmp name	EIP=name
call name	SP=SP-4; M[SP]=EIP+5; EIP=name;
movw EBX,[EDI+45]	EBX=M[EDI+45]
push ESI	SP=SP-4; M[SP]=ESI
pop EDI	EDI=M[SP]; SP=SP+4
add EAX,#6765	EAX= EAX+6765
test EDX,#42	Set condition code (flags) with EDX and 42
movsl	M[EDI]=M[ESI]; EDI=EDI+4; ESI=ESI+4

FIGURE 2.36 Some typical x86 instructions and their functions. A list of frequent operations appears in Figure 2.37. The CALL saves the EIP of the next instruction on the stack. (EIP is the Intel PC.)

Instruction	Meaning
Control	Conditional and unconditional branches
jnz, jz	Jump if condition to EIP + 8-bit offset; JNE (for JNZ), JE (for JZ) are alternative names
jmp	Unconditional jump—8-bit or 16-bit offset
call	Subroutine call—16-bit offset; return address pushed onto stack
ret	Pops return address from stack and jumps to it
loop	Loop branch—decrement ECX; jump to EIP + 8-bit displacement if ECX≠0
Data transfer	Move data between registers or between register and memory
move	Move between two registers or between register and memory
push, pop	Push source operand on stack; pop operand from stack top to a register
les	Load ES and one of the GPRs from memory
Arithmetic, logical	Arithmetic and logical operations using the data registers and memory
add, sub	Add source to destination; subtract source from destination; register-memory format
cmp	Compare source and destination; register-memory format
shl, shr, rcr	Shift left; shift logical right; rotate right with carry condition code as fill
cbw	Convert byte in eight rightmost bits of EAX to 16-bit word in right of EAX
test	Logical AND of source and destination sets condition codes
inc, dec	Increment destination, decrement destination
or, xor	Logical OR; exclusive OR; register-memory format
String	Move between string operands; length given by a repeat prefix
movs	Copies from string source to destination by incrementing ESI and EDI; may be repeated
lods	Loads a byte, word, or doubleword of a string into the EAX register

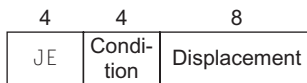
FIGURE 2.37 Some typical operations on the x86. Many operations use register-memory format, where either the source or the destination may be memory and the other may be a register or immediate operand.

x86 Instruction Encoding

Saving the worst for last, the encoding of instructions in the 80386 is complex, with many different instruction formats. Instructions for the 80386 may vary from 1 byte, when there are no operands, up to 15 bytes.

Figure 2.38 shows the instruction format for several of the example instructions in Figure 2.36. The opcode byte usually contains a bit saying whether the operand is 8 bits or 32 bits. For some instructions, the opcode may include the addressing mode and the register; this is true in many instructions that have the form “register = register op

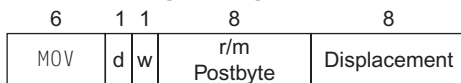
a. JE EIP + displacement



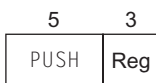
b. CALL



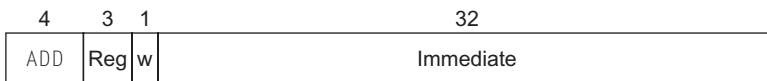
c. MOV EBX, [EDI + 45]



d. PUSH ESI



e. ADD EAX, #6765



f. TEST EDX, #42



FIGURE 2.38 Typical x86 instruction formats. Figure 2.39 shows the encoding of the postbyte. Many instructions contain the 1-bit field *w*, which says whether the operation is a byte or a doubleword. The *d* field in MOV is used in instructions that may move to or from memory and shows the direction of the move. The ADD instruction requires 32 bits for the immediate field, because in 32-bit mode, the immediates are either 8 bits or 32 bits. The immediate field in the TEST is 32 bits long because there is no 8-bit immediate for test in 32-bit mode. Overall, instructions may vary from 1 to 15 bytes in length. The long length comes from extra 1-byte prefixes, having both a 4-byte immediate and a 4-byte displacement address, using an opcode of 2 bytes, and using the scaled index mode specifier, which adds another byte.

reg	w = 0		w = 1		r/m	mod = 0		mod = 1		mod = 2		mod = 3
			16b	32b		16b	32b	16b	32b	16b	32b	
0	AL	AX	EAX	0	addr=BX+SI	=EAX	same	same	same	same		same
1	CL	CX	ECX	1	addr=BX+DI	=ECX	addr as	addr as	addr as	addr as		as
2	DL	DX	EDX	2	addr=BP+SI	=EDX	mod=0	mod=0	mod=0	mod=0		reg
3	BL	BX	EBX	3	addr=BP+SI	=EBX	+ disp8	+ disp8	+ disp16	+ disp32		field
4	AH	SP	ESP	4	addr=SI	=(sib)	SI+disp8	(sib)+disp8	SI+disp8	(sib)+disp32		"
5	CH	BP	EBP	5	addr=DI	=disp32	DI+disp8	EBP+disp8	DI+disp16	EBP+disp32		"
6	DH	SI	ESI	6	addr=disp16	=ESI	BP+disp8	ESI+disp8	BP+disp16	ESI+disp32		"
7	BH	DI	EDI	7	addr=BX	=EDI	BX+disp8	EDI+disp8	BX+disp16	EDI+disp32		"

FIGURE 2.39 The encoding of the first address specifier of the x86: *mod*, *reg*, *r/m*. The first four columns show the encoding of the 3-bit *reg* field, which depends on the *w* bit from the opcode and whether the machine is in 16-bit mode (8086) or 32-bit mode (80386). The remaining columns explain the *mod* and *r/m* fields. The meaning of the 3-bit *r/m* field depends on the value in the 2-bit *mod* field and the address size. Basically, the registers used in the address calculation are listed in the sixth and seventh columns, under *mod* = 0, with *mod* = 1 adding an 8-bit displacement and *mod* = 2 adding a 16-bit or 32-bit displacement, depending on the address mode. The exceptions are 1) *r/m* = 6 when *mod* = 1 or *mod* = 2 in 16-bit mode selects BP plus the displacement; 2) *r/m* = 5 when *mod* = 1 or *mod* = 2 in 32-bit mode selects EBP plus displacement; and 3) *r/m* = 4 in 32-bit mode when *mod* does not equal 3, where (sib) means use the scaled index mode shown in Figure 2.35. When *mod* = 3, the *r/m* field indicates a register, using the same encoding as the *reg* field combined with the *w* bit.

immediate.” Other instructions use a “postbyte” or extra opcode byte, labeled “*mod*, *reg*, *r/m*,” which contains the addressing mode information. This postbyte is used for many of the instructions that address memory. The base plus scaled index mode uses a second postbyte, labeled “*sc*, *index*, *base*.”

Figure 2.39 shows the encoding of the two postbyte address specifiers for both 16-bit and 32-bit modes. Unfortunately, to understand fully which registers and which addressing modes are available, you need to see the encoding of all addressing modes and sometimes even the encoding of the instructions.

x86 Conclusion

Intel had a 16-bit microprocessor two years before its competitors’ more elegant architectures, such as the Motorola 68000, and this head start led to the selection of the 8086 as the CPU for the IBM PC. Intel engineers generally acknowledge that the x86 is more difficult to build than computers like ARMv7 and MIPS, but the large market meant in the PC era that AMD and Intel could afford more resources to help overcome the added complexity. What the x86 lacks in style, it rectifies with market size, making it beautiful from the right perspective.

Its saving grace is that the most frequently used x86 architectural components are not too difficult to implement, as AMD and Intel have demonstrated by rapidly improving performance of integer programs since 1978. To get that performance, compilers must avoid the portions of the architecture that are hard to implement fast.

In the post-PC era, however, despite considerable architectural and manufacturing expertise, x86 has not yet been competitive in the personal mobile device.

2.19 Real Stuff: The Rest of the ARMv8 Instruction set

Figure 2.40 suggests the size of the full ARMv8 architecture by listing the number of assembly language instructions and machine language instructions in each class of instruction types. Chapter 3 describes the floating point, multiply–divide, and SIMD portion of the architecture and Chapter 5 covers the system instructions. In this section, we highlight the integer operations and branches.

One reason for the much larger number of machine instructions than assembly instructions is that ARMv8 includes both 32-bit and 64-bit versions of instructions within the same architecture. In assembly language, programmers use registers named W0, W1, ... instead of the X0, X1, ... to specify 32-bit operations. Thus, this 64-bit operation

```
ADD    X9, X21, X9
```

can be turned into a 32-bit operation by writing instead

```
ADD    W9, W21, W9
```

Figure 2.40 counts this as one assembly language instruction but two machine language instructions since they have different opcodes.

Our goal for the rest of this section is to explain any remaining instructions in ARMv8. It is also to give insight into how the 36 LEGv8 instructions in Figure 2.1 increase to 146 in the full ARMv8 instruction set: 123 assembly language instructions in the first row in Figure 2.40 plus 23 branch instructions in the second to last row.

Class	Loads/Stores		Operations		Branches		Total	
	AL	ML	AL	ML	AL	ML	AL	ML
Integer	49	145	74	105	—	—	123	250
Floating Point & Int Mul/Div	0	18	63	156	—	—	63	174
SIMD/Vector	16	166	229	371	—	—	245	537
System/Special	11	55	52	40	—	—	63	95
—	—	—	—	—	23	14	23	14
Total	76	384	418	672	23	14	517	1070

FIGURE 2.40 The number of instructions of each type and for each function, counted separately for assembly language instructions (AL) and machine language instructions (ML). The counts were based on evaluating the *ARM Architecture Reference Manual: ARMv8, for ARMv8-A architecture profile*, beta edition, 2014. There were also a few minor adjustments made to match the version of the assembly language instructions in this book. For example, this book shows different assembly language instructions for single and precision floating-point arithmetic (FADDs, FADD, ...) but ARMv8 actually uses just has one version (FADD), using the register names to select the opcode.

Full ARMv8 Integer Arithmetic Logic Instructions

A unique feature of the ARMv8 instruction set is that the second register of all arithmetic and logical processing operations has the option of being shifted before being operated on. The shift options are shift left logical, shift right logical, shift right arithmetic, and rotate right. (For the 12-bit immediate instructions, the only shift option is a logical left shift by 12 bits.) Although the assembler has explicit instructions with these names (LSL, LSR, SRA, and ROR), these are really just pseudoinstructions of a regular instruction that only performs the shift. We picked versions of the instructions that put 0 in the shift amount field so that the second operand is unchanged in the LEGv8, but the full instruction set allows shifts as part of arithmetic and logic instructions.

Figure 2.41 lists all 74 assembly language instructions for the integer operations of the full ARMv8 instruction set. We start with the 20 integer operations in the LEGv8 in Figure 2.1. First, the total count of assembly language instructions includes pseudoinstructions from above—CMP, CMPI, MOV—so that bumps the count to 23. There are also five pseudoinstructions based on the instructions in the LEGv8—negate, negate and set flags, compare negative, compare negative immediate, and test immediate—which brings us to 28.

To support arithmetic on narrower data types, there are another six instructions that let you mix data sizes of the second operand by either sign extending it or zero extending it to the full width. The *extended-register* instructions work with bytes, halfwords, or words, which gets us to 34 assembly language instructions.

To support add and subtract operations on operands larger than one doubleword, ARM includes six more assembly instructions to add or subtract the carry from a previous operation. Thus, the first instruction does the regular add of the registers with the right half of the operands and sets the condition codes (ADDS), and the second one does add with the carry out from that operation when adding the registers with the left half of the operands (ADC). These six bring the total to 40 instructions.

The logical instructions (AND, ANDS, ORR, EOR) also have versions where the second operand is complemented: that is, every 0 becomes 1 and vice versa. Including new pseudoinstructions, this variation leads to six more instructions, or a total now of 46.

In addition to the logical shift left and logical shift right instructions, ARMv8 includes two more, as mentioned above. ASR does arithmetic shift right, which replicates the sign bit during the shift, and ROR rotates the bits to the right; that is, the bits shift off to the right are inserted on the left. Although we didn't list them in LEGv8, there are versions of all the shift instructions that determine the amount to be shifted based on a value in a register rather than as an immediate within the instruction. These shifts add six more instructions, bringing us to 52.

There is also a version of the MOV wide instructions (MOVN) that complements all 64 bits that are created from the 16-bit constant; that is, the other 48 bits are ones instead of zeros and the 16-bit immediate field is complemented too, which bumps the count to 53.

To manipulate fields of bits, the full ARMv8 instruction set includes 10 instructions that can extract a bit field from a register and insert it into another.

Type	Mnemonic	Instruction	Type	Mnemonic	Instruction
Arithmetic Register	ADD	Add	Logical Immediate	ANDI	Bitwise AND Immediate
	ADDs	Add and set flags		ANDIS	Bitwise AND and set flags Immediate
	SUB	Subtract		ORRI	Bitwise inclusive OR Immediate
	SUBs	Subtract and set flags		EORI	Bitwise exclusive OR Immediate
	CMP	Compare		<i>TSTI</i>	Test bits Immediate
	<i>CMN</i>	Compare negative		LSL	Logical shift left Immediate
	<i>NEG</i>	Negate	Shift Register Shift Immed	LSR	Logical shift right Immediate
	<i>NEGS</i>	Negate and set flags		ASR	Arithmetic shift right Immediate
Arithmetic Immediate	ADDI	Add Immediate		ROR	Rotate right Immediate
	ADDIS	Add and set flags Immediate		LSRV	Logical shift right register
	SUBI	Subtract Immediate		LSLV	Logical shift left register
	SUBIS	Subtract and set flags Immediate		ASRV	Arithmetic shift right register
	CMPI	Compare Immediate		RORV	Rotate right register
	<i>CMNI</i>	Compare negative Immediate	Move Wide Immed iate	MOVZ	Move wide with zero
Arithmetic Extended	ADD	Add Extended Register		MOVK	Move wide with keep
	ADDs	Add and set flags Extended		MOVN	Move wide with NOT
	SUB	Subtract Extended Register		MOV	Move register
	SUBs	Subtract and set flags Extended	Bit Field Insert & Extract	BFM	Bitfield move
	<i>CMP</i>	Compare Extended Register		SBFM	Signed bitfield move
	<i>CMN</i>	Compare negative Extended		UBFM	Unsigned bitfield move (32-bit)
Arithmetic with Carry	ADC	Add with carry		BFI	Bitfield insert
	ADCS	Add with carry and set flags		BFXIL	Bitfield extract and insert low
	SBC	Subtract with carry		SBFIZ	Signed bitfield insert in zero
	SBCS	Subtract with carry and set flags		SBFX	Signed bitfield extract
	<i>NGC</i>	Negate with carry		UBFIZ	Unsigned bitfield insert in zero
	<i>NGCS</i>	Negate with carry and set flags		UBFX	Unsigned bitfield extract
Logical Register	AND	Bitwise AND	Sign Extend	EXTR	Extract register from pair
	ANDS	Bitwise AND and set flags		<i>SXTB</i>	Sign-extend byte
	ORR	Bitwise inclusive OR		<i>SXTH</i>	Sign-extend halfword
	EOR	Bitwise exclusive OR		<i>SXTW</i>	Sign-extend word
	BIC	Bitwise bit clear		<i>UXTB</i>	Unsigned extend byte
	BICS	Bitwise bit clear and set flags		<i>UXTH</i>	Unsigned extend halfword
	ORN	Bitwise inclusive OR NOT	Bit Operation	CLS	Count leading sign bits
	EON	Bitwise exclusive OR NOT		CLZ	Count leading zero bits
	<i>MVN</i>	Bitwise NOT		RBIT	Reverse bit order
	<i>TST</i>	Test bits		REV	Reverse bytes in register
				REV16	Reverse bytes in halfwords
				REV32	Reverses bytes in words

FIGURE 2.41 The list of assembly language instructions for the integer operations in the full ARMv8 instruction set. Bold means the instruction is also in LEGv8, italic means it is a pseudoinstruction, and bold italic means it is a pseudoinstruction that is also in LEGv8.

It uses those instructions to create five more pseudoinstructions that sign extend or zero extend narrow operands to the full width of a register. Finally, there are another six “bit twiddling” instructions that count leading zeros or ones and reverse order of bits and bytes. The total is now 74 integer operations in the full ARMv8 integer operation assembly language.

Full ARMv8 Integer Data Transfer Instructions

We did not show all of the addressing modes available for the 10 integer data transfer instructions in LEGv8. The addressing mode we described earlier is called *unscaled signed immediate offset* in ARMv8 lingo. Here are five more:

1. Base plus a scaled 12-bit unsigned immediate offset.
2. Base plus a 64-bit register offset, optionally scaled.
3. Base plus a 32-bit extended register offset, optionally scaled.
4. Pre-indexed by an unscaled 9-bit signed immediate offset.
5. Post-indexed by an unscaled 9-bit signed immediate offset.

The scaling options of the first three addressing modes multiply or scale the address in the immediate field or in the register by the size of the data being transferred in bytes. Since you would normally want that address to be a multiple of the data size, these addressing modes are built into the hardware so the programmer doesn't have to convert an index in a register into a byte address. They also increase the range of what the immediate field can represent within an instruction. Thus, if X11 contains $100,000_{\text{ten}}$

```
LDR X10, [X11, #16] // scaled addressing mode
```

will load the double word (8 bytes) at address $100,128_{\text{ten}}$ ($100,000 + 8 \times 16$) into register X10.

The address of the second addressing mode is simply the sum of two registers, with the option of shifting the second operand by 1, 2, or 3 bits, which multiplies the value by 2, 4, or 8. Thus, if X11 contains $100,000_{\text{ten}}$ and X12 contains $1,000_{\text{ten}}$

```
LDR X10, [X11, X12 LSL #3] // base + register, scaled
```

will load the double word (8 bytes) at address $108,000_{\text{ten}}$ ($100,000 + 2^3 \times 1000$) into register X10. The third addressing mode simply uses a 32-bit register (e.g., W12) instead of a 64-bit register (e.g., X12), with the same option of scaling.

The last two addressing modes *change the base register* as part of the address calculation. The use case is when marching through a sequential array, the address increment can happen as part of the addressing mode rather than as part of a separate add or subtract instruction. Thus, if X11 contains $100,000_{\text{ten}}$

```
LDR X10, [X11, #16]! // pre-indexed addressing mode
```

will load the double word (8 bytes) at address $100,016_{\text{ten}}$ into register X10 and change X11 to $100,016_{\text{ten}}$. On the other hand,

```
LDR X10, [X11],#16 // post-indexed addressing mode
```

will load the double word (8 bytes) at address 100,000_{ten} into register X10 and change X11 to 100,016_{ten}. The pre- and post- name indicates whether the address arithmetic occurs before the operand is accessed or afterwards.

The main reason there are so many more machine data transfer instructions (125) than assembly data transfer instructions (49) for integers is that the addressing modes require separate opcodes in machine language, but as we see above, they are just a different addressing notation for the same mnemonics in assembly language.

Figure 2.42 lists the full set of integer data transfer instructions for ARMv8. To get from 10 integer data-transfer instructions to the ARMv8 total of 49, we first add the LDA pseudoinstruction, to get us to 11. Next, we note that there are signed data transfer versions of the *load byte* (LDURSB) and *load halfword* (LDURSH), which brings us to 13. In addition to the doubleword load exclusive and store exclusive instructions, ARMv8 includes exclusive data transfers of bytes, halfwords, and paired double words, yielding another six instructions, or a total of 19.

Type	Mnemonic	Instruction	Type	Mnemonic	Instruction
Unscaled	LDUR	Load register (unscaled offset)	Exclusive	LDXR	Load Exclusive register
	LDURB	Load byte (unscaled offset)		LDXRB	Load Exclusive byte
	LDURSB	Load signed byte (unscaled offset)		LDXRH	Load Exclusive halfword
	LDURH	Load halfword (unscaled offset)		LDXP	Load Exclusive Pair
	LDURSH	Load signed halfword (unscaled offset)		STXR	Store Exclusive register
	LDURSW	Load signed word (unscaled offset)		STXRB	Store Exclusive byte
	STUR	Store register (unscaled offset)		STXRH	Store Exclusive halfword
	STURB	Store byte (unscaled offset)		STXP	Store Exclusive Pair
	STURH	Store halfword (unscaled offset)		LDAXR	Load-acquire Exclusive register
	STURW	Store word (unscaled offset)		LDAXRB	Load-acquire Exclusive byte
	LDA	Load address		LDAXRH	Load-acquire Exclusive halfword
Scaled, Extended, Pre- & Post-Indexed	LDR	Load register	Exclusive Acquire/Release	LDAXP	Load-acquire Exclusive Pair
	LDRB	Load byte		STLXR	Store-release Exclusive register
	LDRSB	Load signed byte		STLXRB	Store-release Exclusive byte
	LDRH	Load halfword		STLXRH	Store-release Exclusive halfword
	LDRSH	Load signed halfword		STLXP	Store-release Exclusive Pair
	LDRSW	Load signed word		LDP	Load Pair
	STR	Store register		LDPSW	Load Pair signed words
	STRB	Store byte		STP	Store Pair
	STRH	Store halfword		ADRP	Compute address of 4KB page at a PC-relative offset
				ADR	Compute address of label at a PC-relative offset
			PC		

FIGURE 2.42 The list of assembly language instructions for the integer data transfer operations in the full ARMv8 instruction set. Bold means the instruction is also in LEGv8, italic means it is a pseudoinstruction, and bold italic means it is a pseudoinstruction that is also in LEGv8.

The data transfers with the other addressing modes use a different mnemonic (LDR, STR, ...), which leads to another nine instructions to get the various sizes both signed and unsigned. To accelerate data transfers, ARMv8 includes three load pair and store pair instructions, which transfer two doublewords at a time. To create PC-relative addresses that can be passed as arguments, ARMv8 has two instructions that add a 21-bit immediate in the current instruction to the current value of the PC and then load it (or the value shifted left by 12 bits) into a register. These 14 instructions bring us to 33.

The final set of data transfer instructions for integers perform exclusive access to memory in multiprocessor environments. We saw two examples earlier in LDXR and STXR. In addition to providing exclusive access to doublewords, there are versions for bytes, halfwords, and paired registers. ARMv8 also offers load acquire and store release versions, which implement the release-consistency memory model to support programming language standards for shared memory semantics. This set adds 16 exclusive data transfer instructions in the second column of Figure 2.42, bringing us to the final total of 49 data transfer instructions.

Full ARMv8 Branch Instructions

Figure 2.43 lists all the branch instructions for ARMv8. To get from six branch instructions in LEGv8 to the ARMv8 total of 23, we start with two more conditional branch instructions that test to see if a bit within a register is zero or one. There are also two more unconditional branches. The first is a variation of branch and link that uses a register for the *branch address* (BALR). The second is return from *subroutine* (RET), which sounds a lot like *branch register* (BR); the reason they sound similar is that they have the identical semantics. The reason ARMv8 has different opcodes for the same operation is so that hardware branch predictors (Chapter 4) can know whether it is really return from a *subroutine* (RET), which is easy to predict, or

Type	Mnemonic	Instruction	Type	Mnemonic	Instruction
Conditional Branch	B . cond	Branch conditionally	Conditional Select	CSEL	Conditional select
	CBNZ	Compare and branch if nonzero		CSINC	Conditional select increment
	CBZ	Compare and branch if zero		CSINV	Conditional select inversion
	TBNZ	Test bit and branch if nonzero		CSNEG	Conditional select negation
	TBZ	Test bit and branch if zero		<i>CSET</i>	Conditional set
Unconditional Branch	B	Branch unconditionally		<i>CSETM</i>	Conditional set mask
	BL	Branch with link		<i>CINC</i>	Conditional increment
	BLR	Branch with link to register		<i>CINV</i>	Conditional invert
	BR	Branch to register		<i>CNEG</i>	Conditional negate
	RET	Return from subroutine	Conditional Compare	CCMP	Conditional compare register
				CCMPI	Conditional compare immediate
				CCMN	Conditional compare negative register
				CCMNI	Conditional compare negative immediate

FIGURE 2.43 The list of assembly language instructions for the branches of the ARMv8 instruction set. Bold means the instruction is also in LEGv8, italic means it is a pseudoinstruction, and bold italic means it is a pseudoinstruction that is also in LEGv8.

being used in a *branch table* (BR), which is much harder to predict and should use a distinct mechanism. These additions get us to 10 branches.

The next nine instructions store a value into a register based on the condition codes. If the condition is true, the destination register gets the first register. If not, it gets the second register. The idea behind condition select instructions is to replace conditional branches, which can cause problems in pipelined execution if they can't be predicted (see [Chapter 4](#)). After adding the nine condition select instructions, we're up to 19.

The final four instructions are similar to the conditional select instructions, except the destination for these instructions is the condition codes. That is, these instructions conditionally set the condition codes based on the current condition codes, which is a bit confusing. If the selected condition is true, then the condition codes are set by the result of an arithmetic comparison of two operands, or to an immediate value if the condition is false. One version of conditional compare instruction checks two registers, and the other compares a register to a small 5-bit unsigned value. These last four instructions raise the total to 23 branch instructions in ARMv8.

2.20 Fallacies and Pitfalls

Fallacy: More powerful instructions mean higher performance.

Part of the power of the Intel x86 is the prefixes that can modify the execution of the following instruction. One prefix can repeat the subsequent instruction until a counter steps down to 0. Thus, to move data in memory, it would seem that the natural instruction sequence is to use move with the repeat prefix to perform 32-bit memory-to-memory moves.

An alternative method, which uses the standard instructions found in all computers, is to load the data into the registers and then store the registers back to memory. This second version of this program, with the code replicated to reduce loop overhead, copies at about 1.5 times as fast. A third version, which uses the larger floating-point registers instead of the integer registers of the x86, copies at about 2.0 times as fast as the complex move instruction.

Fallacy: Write in assembly language to obtain the highest performance.

At one time compilers for programming languages produced naïve instruction sequences; the increasing sophistication of compilers means the gap between compiled code and code produced by hand is closing fast. In fact, to compete with current compilers, the assembly language programmer needs to understand the concepts in Chapters 4 and 5 thoroughly (processor pipelining and memory hierarchy).

This battle between compilers and assembly language coders is another situation in which humans are losing ground. For example, C offers the programmer a chance to give a hint to the compiler about which variables to keep in registers versus spilled to memory. When compilers were poor at register allocation, such hints were vital to performance. In fact, some old C textbooks spent a fair amount of time giving examples that effectively use register hints. Today's C compilers generally ignore these hints, because the compiler does a better job at allocation than the programmer does.

Even *if* writing by hand resulted in faster code, the dangers of writing in assembly language are the protracted time spent coding and debugging, the loss in portability, and the difficulty of maintaining such code. One of the few widely accepted axioms of software engineering is that coding takes longer if you write more lines, and it clearly takes many more lines to write a program in assembly language than in C or Java. Moreover, once it is coded, the next danger is that it will become a popular program. Such programs always live longer than expected, meaning that someone will have to update the code over several years and make it work with new releases of operating systems and recent computers. Writing in higher-level language instead of assembly language not only allows future compilers to tailor the code to forthcoming machines; it also makes the software easier to maintain and allows the program to run on more brands of computers.

Fallacy: The importance of commercial binary compatibility means successful instruction sets don't change.

While backwards binary compatibility is sacrosanct, [Figure 2.44](#) shows that the x86 architecture has grown dramatically. The average is more than one instruction per month over its 35-year lifetime!

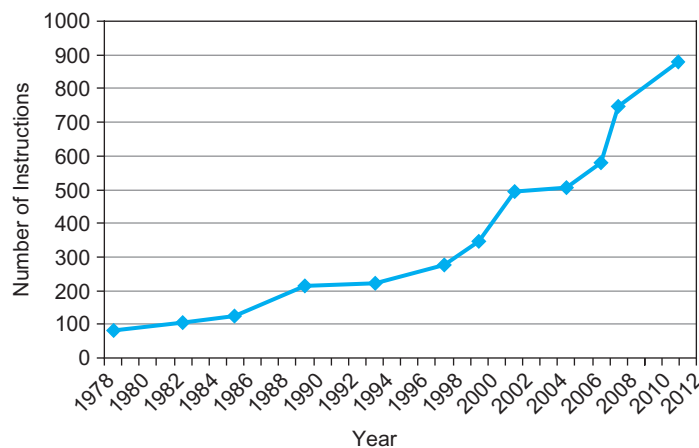


FIGURE 2.44 Growth of x86 instruction set over time. While there is clear technical value to some of these extensions, this rapid change also increases the difficulty for other companies to try to build compatible processors.

Pitfall: Forgetting that sequential word or doubleword addresses in machines with byte addressing do not differ by one.

Many an assembly language programmer has toiled over errors made by assuming that the address of the next word or doubleword can be found by incrementing the address in a register by one instead of by the word or doubleword size in bytes. Forewarned is forearmed!

Pitfall: Using a pointer to an automatic variable outside its defining procedure.

A common mistake in dealing with pointers is to pass a result from a procedure that includes a pointer to an array that is local to that procedure. Following the stack discipline in [Figure 2.13](#), the memory that contains the local array will be reused as soon as the procedure returns. Pointers to automatic variables can lead to chaos.

2.21 Concluding Remarks

The two principles of the *stored-program* computer are the use of instructions that are indistinguishable from numbers and the use of alterable memory for programs. These principles allow a single machine to aid cancer researchers, financial advisers, and novelists in their specialties. The selection of a set of instructions that the machine can understand demands a delicate balance among the number of instructions needed to execute a program, the number of clock cycles needed by an instruction, and the speed of the clock. As illustrated in this chapter, three design principles guide the authors of instruction sets in making that tricky tradeoff:

1. *Simplicity favors regularity.* Regularity motivates many features of the ARMv8 instruction set: keeping all instructions a single size, always requiring three register operands in arithmetic instructions, and keeping the register fields in the same place in most instruction formats.
2. *Smaller is faster.* The desire for speed is the reason that ARMv8 has 32 registers rather than many more.
3. *Good design demands good compromises.* One ARMv8 example was the compromise between providing for larger addresses and constants in instructions and keeping all instructions the same length.

We also saw the great idea from Chapter 1 of making the **common cast fast** applied to instruction sets as well as computer architecture. Examples of making the common ARMv8 case fast include PC-relative addressing for conditional branches and immediate addressing for larger constant operands.

Above this machine level is assembly language, a language that humans can read. The assembler translates it into the binary numbers that machines can understand,

Less is more.

Robert Browning,
Andrea del Sarto, 1855



COMMON CASE FAST

and it even “extends” the instruction set by creating symbolic instructions that aren’t in the hardware. For instance, constants or addresses that are too big are broken into properly sized pieces, common variations of instructions are given their own name, and so on. [Figure 2.45](#) lists the LEGv8 instructions we have covered so far,

LEGv8 instructions	Name	Format	Pseudo LEGv8	Name	Format
add	ADD	R	move	MOV	R
subtract	SUB	R	compare	CMP	R
add immediate	ADDI	I	compare immediate	CMPI	I
subtract immediate	SUBI	I	load address	LDA	M
add and set flags	ADDS	R			
subtract and set flags	SUBS	R			
add immediate and set flags	ADDIS	I			
subtract immediate and set flags	SUBIS	I			
load register	LDUR	D			
store register	STUR	D			
load signed word	LDURSW	D			
store word	STURW	D			
load half	LDURH	D			
store half	STURH	D			
load byte	LDURB	D			
store byte	STURB	D			
load exclusive register	LDXR	D			
store exclusive register	STXR	D			
move wide with zero	MOVZ	IM			
move wide with keep	MOVK	IM			
and	AND	R			
inclusive or	ORR	R			
exclusive or	EOR	R			
and immediate	ANDI	I			
inclusive or immediate	ORRI	I			
exclusive or immediate	EORI	I			
logical shift left	LSL	R			
logical shift right	LSR	R			
compare and branch on equal 0	CBZ	CB			
compare and branch on not equal 0	CBNZ	CB			
branch conditionally	B, cond	CB			
branch	B	B			
branch to register	BR	R			
branch with link	BL	B			

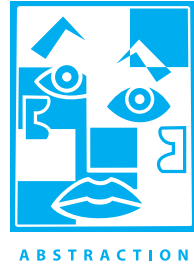
FIGURE 2.45 The LEGv8 instruction set covered so far, with the real LEGv8 instructions on the left and the pseudoinstructions on the right. Section 2.19 describes the full ARMv8 architecture. [Figure 2.1](#) shows more details of the LEGv8 architecture revealed in this chapter. The information given here is also found in Columns 1 and 2 of the LEGv8 Reference Data Card at the front of the book.

both real and pseudoinstructions. Hiding details from the higher level is another example of the great idea of **abstraction**.

Each category of ARMv8 instructions is associated with constructs that appear in programming languages:

- Arithmetic instructions correspond to the operations found in assignment statements.
- Transfer instructions are most likely to occur when dealing with data structures like arrays or structures.
- Conditional branches are used in *if* statements and in loops.
- Unconditional branches are used in procedure calls and returns and for *case/switch* statements.

These instructions are not born equal; the popularity of the few dominates the many. For example, [Figure 2.46](#) shows the popularity of each class of instructions for SPEC CPU2006. The varying popularity of instructions plays an important role in the chapters about datapath, control, and pipelining.



Instruction class	LEGv8 examples	HLL correspondence	Frequency	
			Integer	Ft. pt.
Arithmetic	ADD, SUB, ADDI, SUBI	Operations in assignment statements	16%	48%
Data transfer	LDUR, STUR, LDURSW, STURW, LDURH, STURH, LDURB, STURB, MOVZ, MOVK	References to data structures, such as arrays	35%	36%
Logical	AND, ORR, EOR, ANDI, ORRI, EORI, LSL, LSR	Operations in assignment statements	12%	4%
Conditional branch	CBZ, CBNZ, B.cond, CMP, CMPI	<i>If</i> statements and loops	34%	8%
Jump	B, BR, BL	Procedure calls, returns, and <i>case/switch</i> statements	2%	0%

FIGURE 2.46 LEGv8 instruction classes, examples, correspondence to high-level program language constructs, and percentage of LEGv8 instructions executed by category for the average integer and floating point SPEC CPU2006 benchmarks. [Figure 3.27](#) in [Chapter 3](#) shows average percentage of the individual LEGv8 instructions executed.

After we explain computer arithmetic in [Chapter 3](#), we reveal more of the ARMv8 instruction set architecture.



Historical Perspective and Further Reading

This section surveys the history of *instruction set architectures* (ISAs) over time, and we give a short history of programming languages and compilers. ISAs include accumulator architectures, general-purpose register architectures, stack



Historical Perspective and Further Reading

This section surveys the history of instruction set architectures over time, and we give a short history of programming languages and compilers. ISAs include accumulator architectures, general-purpose register architectures, stack architectures, and a brief history of ARMv7 and the x86. We also review the controversial subjects of high-level-language computer architectures and reduced instruction set computer architectures. The history of programming languages includes Fortran, Lisp, Algol, C, Cobol, Pascal, Simula, Smalltalk, C++, and Java, and the history of compilers includes the key milestones and the pioneers who achieved them.

Accumulator Architectures

Hardware was precious in the earliest stored-program computers. Consequently, computer pioneers could not afford the number of registers found in today's architectures. In fact, these architectures had a single register for arithmetic instructions. Since all operations would accumulate in one register, it was called the **accumulator**, and this style of instruction set is given the same name. For example, EDSAC in 1949 had a single accumulator.

The three-operand format of ARMv8 suggests that a single register is at least two registers shy of our needs. Having the accumulator as both a source operand *and* the destination of the operation fills part of the shortfall, but it still leaves us one operand short. That final operand is found in memory. Accumulator architectures have the memory-based operand-addressing mode suggested earlier. It follows that the add instruction of an accumulator instruction set would look like this:

```
ADD      200
```

This instruction means add the accumulator to the word in memory at address 200 and place the sum back into the accumulator. No registers are specified because the accumulator is known to be both a source and a destination of the operation.

The next step in the evolution of instruction sets was the addition of registers dedicated to specific operations. Hence, registers might be included to act as indices for array references in data transfer instructions, to act as separate accumulators for multiply or divide instructions, and to serve as the top-of-stack pointer. Perhaps the best-known example of this style of instruction set is found in the Intel 80x86. This style of instruction set is labeled *extended accumulator*, *dedicated register*, or *special-purpose register*. Like the single-register accumulator architectures, one operand may be in memory for arithmetic instructions. Like the ARMv8 architecture, however, there are also instructions where all the operands are registers.

accumulator Archaic term for register. On-line use of it as a synonym for “register” is a fairly reliable indication that the user has been around quite a while.

Eric Raymond, *The New Hacker's Dictionary*, 1991

load-store architecture Also called **register-register architecture**. An instruction set architecture in which all operations are between registers and data memory may only be accessed via loads or stores.

General-Purpose Register Architectures

The generalization of the dedicated-register architecture allows all the registers to be used for any purpose, hence the name *general-purpose register*. ARMv8 is an example of a general-purpose register architecture. This style of instruction set may be further divided into those that allow one operand to be in memory (as found in accumulator architectures), called a *register-memory* architecture, and those that demand that operands always be in registers, called either a **load-store** or a **register-register** architecture. Figure e2.22.1 shows a history of the number of registers in some popular computers.

The first load-store architecture was the CDC 6600 in 1963, considered by many to be the first supercomputer. ARMv7, ARMv8, and MIPS are more recent examples of a load-store architecture.

Machine	Number of general-purpose registers	Architectural style	Year
EDSAC	1	Accumulator	1949
IBM 701	1	Accumulator	1953
CDC 6600	8	Load-store	1963
IBM 360	16	Register-memory	1964
DEC PDP-8	1	Accumulator	1965
DEC PDP-11	8	Register-memory	1970
Intel 8008	1	Accumulator	1972
Motorola 6800	2	Accumulator	1974
DEC VAX	16	Register-memory, memory-memory	1977
Intel 8086	1	Extended accumulator	1978
Motorola 68000	16	Register-memory	1980
Intel 80386	8	Register-memory	1985
ARM	16	Load-store	1985
MIPS	32	Load-store	1985
HP PA-RISC	32	Load-store	1986
SPARC	32	Load-store	1987
PowerPC	32	Load-store	1992
DEC Alpha	32	Load-store	1992
HP/Intel IA-64	128	Load-store	2001
AMD64 (EMT64)	16	Register-memory	2003
ARMv8	16	Load-store	2013

FIGURE e2.22.1 The number of general-purpose registers in popular architectures over the years.

The 80386 was Intel’s attempt to transform the 8086 into a general-purpose register-memory instruction set. Perhaps the best-known register-memory instruction set is the IBM 360 architecture, first announced in 1964. This instruction set is still at the core of IBM’s mainframe computers—responsible for a large part